

kopForth v0.3

LGP-30 Edition

User Manual

This is a port of the [kopForth v0.3](<https://github.com/koppanyh/kopForth/tree/v0.3>) interpreter made to run on the LGP-30.

Due to major architectural differences between the LGP and modern processors, this isn't an exact 1-to-1 port, but it should be mostly similar. It is also not implementing all of the words found in the official kopForth due to memory constraints.

The source code and compiled tape for this port can be found on [GitHub](<https://github.com/koppanyh/LGP-30/tree/main/kopForth>)

Quickstart

Loading Forth

- 1) Enable **Manual Input**
- 2) Press **Clear Counter**
- 3) Enable **Normal** operation
- 4) Press **Start**
- 5) Send paper tape via terminal
- 6) Enable **6 Bit Input**
- 7) Press **Start**

Refer to [section 3.5.1]

(<https://www.e-basteln.de/file/lgp30/LittleGP-30%20documentation%20-%20wide%20USB.pdf>) of the LittleGP-30 documentation.

Text Entry

Due to the strangeness of how text input works on the LGP-30, text must be entered with no more than 5 characters at a time. This is because LGP text input works by shifting the accumulator left 6 bits, then writes the current 6 bit character input into the now-empty lower 6 bits. This is a hardware behavior. As such, the 31-ish bit words of the LGP can hold 5 6-bit characters each.

The input characters are also limited to the following set:

0123456789fgjkgwzbyridnmpeuthcas +-;/.,vox

With 1 and l being the same character.

And you finish entering sets of characters by terminating with a single quote: '.

There is also no backspace, so don't mess up...

Enter individual words (i.e. tokens) as groups of 5 characters separated by a single space token.

For example, 5 compile, would look like this:

```
5' 'compi'le, ''
```

The empty token at the end tells the interpreter to process the line (substitute for return key).

It's important that larger words be grouped in 5 characters, since

```
compi'le, '
```

Is treated as a different word than

```
comp'ile, '
```

This version of kopForth is case sensitive.

Simh

Build Instructions

Run the `main.py` program which will link all the kopForth libraries together and emit the tape program.

Example command to save to tape file: `python main.py > kopForth.tx`

Run Instructions

Within Simh, run the following:

```
load kopForth.tx
set cpu manual
set cpu 6b
co
```

Alternatively, call Simh with the `kopForth.simh` file as an input.

[Simh builds](<https://github.com/simh/Development-Binaries>) can be found here and contain the `lgp.exe` emulator executable used to develop this port.

Simh presents text input as starting with a backtick: `

So for example, 5 compile, would look like this:

```
`5`` ``compi``le, ``
```

Word Listing

The following words are implemented in this forth. If a word is in the **LGP** column, then use that name when referencing the word. Words that are not in the **Standard** column are specific to this forth implementation. Words with a ***** in the 3rd column are not standard compliant (mostly due to memory or architecture differences).

Standard	LGP		Signature	Description
exit			--	Return from a word
	literal-c		-- n	SYSTEM WORD
-			$n_1 \ n_2 \ -- \ n_3$	Subtract 2 numbers
.			n --	Print a number
@	peek		a -- n	Read the value at an address
!	poke		n a --	Write a value to an address
c@	c-peek	*	a -- n	Same as @
c!	c-poke	*	n a --	Same as !
>r	r-push		n --	Push to return stack from data stack
r>	r-pop		-- n	Pop from return stack to data stack
drop			n --	Remove an item from the stack
dup			n -- n n	Duplicate an item on the stack
swap			$n_1 \ n_2 \ -- \ n_2 \ n_1$	Swap top 2 item positions on the stack
	branch		--	SYSTEM WORD
	0branch		n --	SYSTEM WORD
emit		*	n --	Print packed characters
key		*	-- n	Read packed characters
accept		*	a $n_1 \ -- \ n_2$	Read a sequence of packed characters and store at address
type		*	a n --	Print a sequence of packed characters from address
find			a -- a 0 xt 1 xt -1	Find a word by name
=	eq		$n_1 \ n_2 \ -- \ f$	Compare equality of 2 numbers
<	lss		$n_1 \ n_2 \ -- \ f$	Check if number is less than
	nand		$n_1 \ n_2 \ -- \ n_3$	Bitwise NAND of 2 numbers
	s-quot-c		-- a n	SYSTEM WORD
.s			-- a n	View stack contents

Standard	LGP		Signature	Description
dump			a n --	Dump memory at address
	clr-in-source		--	SYSTEM WORD
	clr-ret-stack		--	SYSTEM WORD
	clr-dat-stack		* --	SYSTEM WORD
	err-not-imp		--	SYSTEM WORD
	err-name-long		--	SYSTEM WORD
cells	num2addr	*	n -- a	Turn a number into an address
	addr2num		a -- n	Turn an address into a number
	halt		--	Triggers a halt condition on the LGP-30
	nodebug		--	Turns off debugging mode
	yesdebug		--	Turns on debugging mode
	dp		-- a	Pointer to HERE variable
	lp		-- a	Pointer to LATEST variable
	pp		-- a	Pointer to PENDING variable
state			-- a	Pointer to STATE variable
	srcpt		-- a	Pointer to SOURCE ID variable
>in	in-ptr		-- a	Pointer to SOURCE offset variable
#tib	tib-len		-- a	Pointer to SOURCE length variable
	tp		-- a	Pointer to SOURCE buffer pointer
	debug		-- a	Pointer to DEBUG variable
here			-- a	Value of HERE variable
latest			-- a	Value of LATEST variable
pad			-- a	Pointer to temporary working space
	tib-end		-- a	Pointer to end of buffer area
true			-- -1	Boolean true
false			-- 0	Boolean false
source-id			-- 0 -1	Value of SOURCE ID variable
tib			-- a	Pointer to SOURCE buffer
	tib-avail		-- n	Size of SOURCE buffer
over			$n_1 \ n_2 \ -- \ n_1 \ n_2$ n_1	Copy n_1 to top of stack
rot			$n_1 \ n_2 \ n_3 \ -- \ n_2$ $n_3 \ n_1$	Rotate positions of top 3 stack items
2drop			$n_1 \ n_2 \ --$	Drop the top 2 stack items

Standard	LGP		Signature	Description
2dup			$n_1 \ n_2 \ -- \ n_1 \ n_2$ $n_1 \ n_2$	Duplicate the top 2 stack items
+			$n_1 \ n_2 \ -- \ n_3$	Add two numbers
invert			$n_1 \ -- \ n_2$	Bitwise NOT
or			$n_1 \ n_2 \ -- \ n_3$	Bitwise OR of 2 numbers
and			$n_1 \ n_2 \ -- \ n_3$	Bitwise AND of 2 numbers
0=	0eq		$n \ -- \ f$	Logical NOT
>	gtr		$n_1 \ n_2 \ -- \ f$	Check if number is greater than
>=	geq		$n_1 \ n_2 \ -- \ f$	Check if number is greater or equal
r@	r-peek		$-- \ n$	Copy top of return stack to data stack
rshift		*	$n_1 \ -- \ n_2$	Shift bits down 1 (divide by 2)
+!	+poke		$n \ a \ --$	Add number to value at address and save to address
allot		*	$n \ --$	Increment HERE by n words
,			$n \ --$	Save n to *HERE and increment HERE
compile,			$xt \ --$	Save xt to *HERE and increment HERE
execute			$xt \ -- \ ?$	Execute word from execution token
move		*	$a_1 \ a_2 \ n \ --$	Move n words from a_1 to a_2
cr			$--$	Print new line
bl		*	$-- \ n$	Put packed characters containing only space onto the stack
space			$--$	Print a space
count			$a_1 \ -- \ a_2 \ n$	Get address and size of string in memory
/string		*	$a_1 \ n_1 \ n \ -- \ a_2$ n_2	Return address and size of string offset by n words
	xt-skip	*	$a_1 \ n_1 \ xt \ -- \ a_2$ n_2	Skip characters identified by xt and return string offset
	spread-chars		$n_1 \ a \ -- \ n_2$	SYSTEM WORD
	is-digit		$n_1 \ -- \ n_2 \ -1 \ $ 0	Parse character to digit if possible
	10*		$n_1 \ -- \ n_2$	Multiply a number by 10
>number	to-number	*	$n_1 \ a_1 \ n_2 \ -- \ n_4$ $a_2 \ n_3$	Parse string to number
	s-to-number		$a_1 \ n_1 \ -- \ n \ 0 \ 0$ $ \ a_2 \ n_2$	Parse string to number, improved
abort			$--$	Warm reset the interpreter

Standard	LGP		Signature	Description
source			-- a n	Get address and size of input buffer
refill			-- f	Read a line into the input buffer
	skipchar		-- a	Pointer to the skipchar variable
	isskipchar		n -- f	True if character equals skipchar
	isnotskipchar		n -- f	True if character not equals skipchar
parse-name			-- a n	Get address and size of next word name in the input buffer
	padword		--	SYSTEM WORD
word			n -- a	Read the next word name from the input buffer and store to HERE
	err-not-found		a --	SYSTEM WORD
[	intp-mode		--	Set interpretation mode
]	comp-mode		--	Set compilation mode
>name	name-ptr		xt -- a	Get NAME pointer of a word
>flags	flags-ptr		xt -- a	Get FLAGS pointer of a word
	err-not-intp		xt --	SYSTEM WORD
	interpret		--	Forth outer interpreter
quit			--	Forth outer interpreter loop
reveal			--	Move a word from PENDING to LATEST
>link	link-ptr		xt -- a	Get LINK pointer of a word
>code	code-ptr		xt -- a	Get CODE pointer of a word
>body	body-ptr		xt -- a	Get BODY pointer of a word
	unreveal		--	Move a word from LATEST to PENDING
does>	does-def		-- a	Define behavior for words defined by this word
create			--	Create an empty word definition at HERE
:	def		--	Start the definition of a word
;			--	End the definition of a word
immediate			--	Make a word execute during compilation
	compile-only		--	Make a word only usable during compilation
if			f --	Start of a conditional
then			--	End of a conditional
else			--	Conditional behavior if false

Standard	LGP		Signature	Description
i			-- n	Get index of a DO loop
do			n_1 n_2 --	Define finite loop, starts at n_2 and ends before n_1 <i>for (int i = n_2; i < n_1; i++) {...}</i>
leave			--	Exit a DO loop (break)
+loop			n --	Increment a DO loop by n
loop			--	Increment a DO loop by 1
begin			--	Define indefinite loop
again			--	End definition and jump to start of indefinite loop <i>while (true) {...}</i>
until			f --	End definition and jump to start of indefinite loop if condition is false <i>do {...} while (!f)</i>
while			f --	Set execution condition for indefinite loop; used in conjunction with REPEAT <i>while (f) {...}</i>
repeat			--	End definition and jump to start of indefinite loop with while
bye			--	Terminate execution of kopForth

Signature keys

n	Number
a	Address
f	Boolean flag
xt	Execution token